

系统开发工具基础

第三次实验报告

Python 打包与智能体编程

丁翊君 学号：25020007016

2026 年 9 月 7 日

目录

1 实验目的	3
2 实验环境	3
3 课上实验内容	3
3.1 第 9 题: 打包 greetlab 并干净环境安装	3
3.1.1 题目要求	3
3.1.2 执行过程与结果	4
3.1.3 关键技术点	5
3.2 第 10 题: 智能体修复循环	6
3.2.1 题目要求	6
3.2.2 执行过程与结果	6
3.2.3 关键技术点	9
3.3 第 11 题: 协作材料改写	10
3.3.1 题目要求	10
3.3.2 执行过程与结果	10
3.3.3 关键技术点	12
3.4 第 12 题: PyTorch 线性回归训练	12
3.4.1 题目要求	12
3.4.2 执行过程与结果	12
3.4.3 关键技术点	14
4 课后练习	14
4.1 打包与发布代码 (2 题)	14
4.2 代理编码 (4 题)	14
4.3 代码之外 (3 题)	15
5 进度说明	15
6 Git 提交记录	15

1 实验目的

本次实验是《系统开发工具基础》课程的第三次实验，主题覆盖 Python 打包分发、智能体编程、协作沟通材料与 PyTorch 基础训练。共四题：

1. **打包 greetlab 并干净环境安装**：用 `pyproject.toml` + `src` 布局构建 wheel，在全新虚拟环境中安装并运行命令行入口。
2. **智能体修复循环**：让编程智能体在失败测试驱动下修改实现，人工检查 diff 后确认修复，并记录 `ai_log.md`。
3. **协作材料改写**：围绕同一缺陷，把低质量的 Issue、提交信息和评审意见改写成专业版本。
4. **PyTorch 线性回归**：补全训练循环，验证最终损失收敛到 0.001 以下。

2 实验环境

实验在课程云电脑的 Ubuntu 环境中完成，只列出实验用到的基础软件：

表 1: 实验环境配置

项目	配置
操作系统	Ubuntu 22.04.3 LTS (64 位)
Shell	GNU Bash
Python	3.12.11
git	2.x
make	GNU Make
pytest	9.1.1
ruff	0.16.6
PyTorch	2.14.0+cpu
XeTeX	TeX Live

3 课上实验内容

3.1 第 9 题：打包 greetlab 并干净环境安装

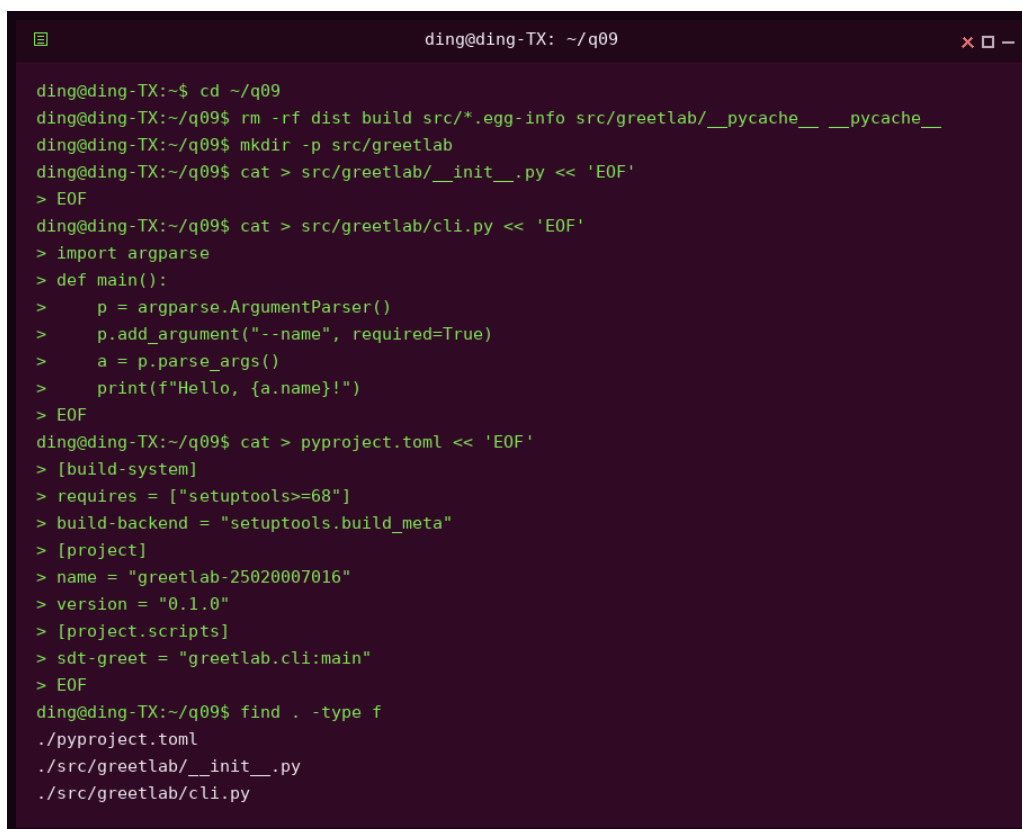
3.1.1 题目要求

在 q09 中创建可安装的 Python 包 `greetlab`，要求：

- 使用 src 布局,包内提供命令行入口 sdt-greet,运行 sdt-greet --name 25020007016 输出问候语。
- 用 python -m build 构建 wheel 包。
- 在全新的虚拟环境中只从 wheel 安装,验证入口可正常运行。

3.1.2 执行过程与结果

步骤 1: 创建包结构。在 q09 下创建 src/greetlab/, 写入 __init__.py 与 cli.py, 命令行入口用 argparse 解析 --name 参数并输出问候语; pyproject.toml 声明构建后端为 setuptools, 并在 [project.scripts] 中注册 sdt-greet = "greetlab.cli:main"。

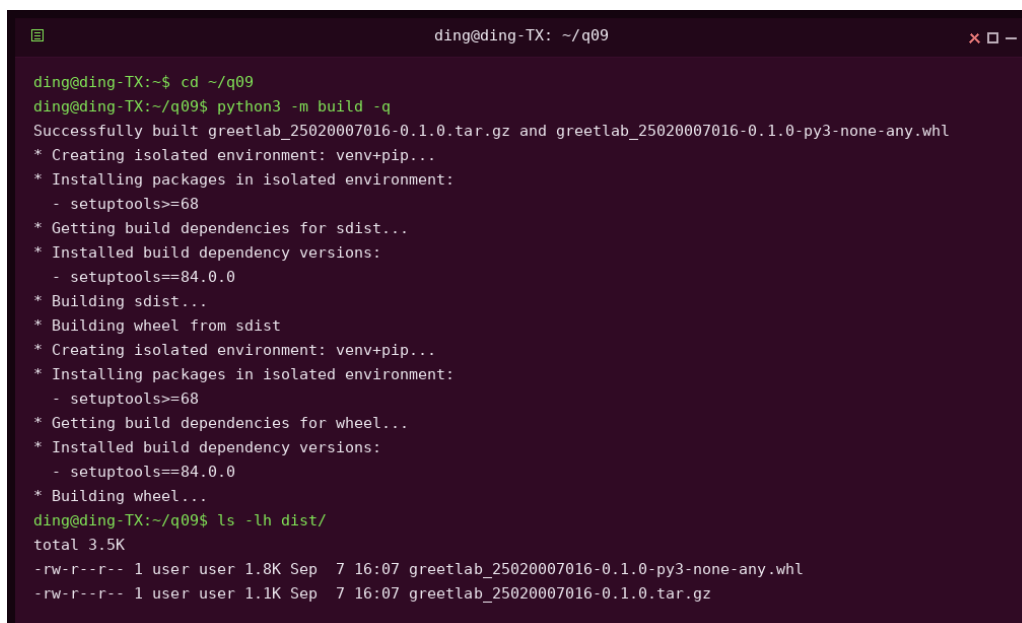


```
ding@ding-TX: ~/q09

ding@ding-TX:~$ cd ~/q09
ding@ding-TX:~/q09$ rm -rf dist build src/*.egg-info src/greetlab/__pycache__ __pycache__
ding@ding-TX:~/q09$ mkdir -p src/greetlab
ding@ding-TX:~/q09$ cat > src/greetlab/__init__.py << 'EOF'
> EOF
ding@ding-TX:~/q09$ cat > src/greetlab/cli.py << 'EOF'
> import argparse
> def main():
>     p = argparse.ArgumentParser()
>     p.add_argument("--name", required=True)
>     a = p.parse_args()
>     print(f"Hello, {a.name}!")
> EOF
ding@ding-TX:~/q09$ cat > pyproject.toml << 'EOF'
> [build-system]
> requires = ["setuptools>=68"]
> build-backend = "setuptools.build_meta"
> [project]
> name = "greetlab-25020007016"
> version = "0.1.0"
> [project.scripts]
> sdt-greet = "greetlab.cli:main"
> EOF
ding@ding-TX:~/q09$ find . -type f
./pyproject.toml
./src/greetlab/__init__.py
./src/greetlab/cli.py
```

图 1: 第 9 题步骤 1: 创建 greetlab 包结构

步骤 2: 构建 wheel。运行 python3 -m build, 成功产出 greetlab_25020007016-0.1.0-py3-none 与对应 sdist。



```
ding@ding-TX: ~/q09
ding@ding-TX:~$ cd ~/q09
ding@ding-TX:~/q09$ python3 -m build -q
Successfully built greetlab_25020007016-0.1.0.tar.gz and greetlab_25020007016-0.1.0-py3-none-any.whl
* Creating isolated environment: venv+pip...
* Installing packages in isolated environment:
  - setuptools>=68
* Getting build dependencies for sdist...
* Installed build dependency versions:
  - setuptools==84.0.0
* Building sdist...
* Building wheel from sdist
* Creating isolated environment: venv+pip...
* Installing packages in isolated environment:
  - setuptools>=68
* Getting build dependencies for wheel...
* Installed build dependency versions:
  - setuptools==84.0.0
* Building wheel...
ding@ding-TX:~/q09$ ls -lh dist/
total 3.5K
-rw-r--r-- 1 user user 1.8K Sep  7 16:07 greetlab_25020007016-0.1.0-py3-none-any.whl
-rw-r--r-- 1 user user 1.1K Sep  7 16:07 greetlab_25020007016-0.1.0.tar.gz
```

图 2: 第 9 题步骤 2: 构建 wheel 与 sdist

步骤 3: 干净环境安装验证。在 q09 外创建全新虚拟环境 q09-venv, 只从 dist/ 下的 wheel 安装; 在包目录之外运行 sdt-greet --name 25020007016, 输出 Hello, 25020007016!, 退出码 0。



```
ding@ding-TX: ~/q09
ding@ding-TX:~$ cd ~/q09
ding@ding-TX:~/q09$ python3 -m venv ~/q09-venv
ding@ding-TX:~/q09$ ~/q09-venv/bin/pip install dist/greetlab_25020007016-0.1.0-py3-none-any.whl -q

[notice] A new release of pip is available: 25.0.1 -> 26.2.1
[notice] To update, run: python3 -m pip install --upgrade pip
ding@ding-TX:~/q09$ cd ~
ding@ding-TX:~$ ~/q09-venv/bin/sdt-greet --name 25020007016
Hello, 25020007016!
```

图 3: 第 9 题步骤 3: 干净 venv 安装 wheel 并运行

3.1.3 关键技术点

1. **src 布局:** 源码放在 src/greetlab/ 下, 构建时打包的是 src 内内容, 避免把仓库根目录的杂项文件装进包里。
2. **[project.scripts] 入口:** 声明 sdt-greet = "greetlab.cli:main" 后, 安装 wheel 会自动生成可执行脚本, 调用包内 main()。
3. **干净环境验证:** python -m venv 创建隔离环境, 只安装 wheel 后运行, 证明包本身不依赖开发环境里的任何隐式状态。

3.2 第 10 题：智能体修复循环

3.2.1 题目要求

复用第 9 题的项目，在 q10 中完成一次智能体驱动的修复：

- 添加一个失败测试：当 `name` 只含空白字符时，`main` 应以 `SystemExit(2)` 结束；先运行测试确认失败。
- 向编程智能体说明目标、约束和测试命令，要求它修改实现并运行测试。
- 人工检查 `git diff`，撤销无关修改，再次运行测试。
- 用不超过 5 行的 `ai_log.md` 记录核心提示、智能体改动和人工验证。

3.2.2 执行过程与结果

步骤 1：添加失败测试并确认失败。复制 q09 为 q10，`git init` 提交基线；编写 `test_cli.py`，用 `subprocess` 运行 `python3 -m greetlab.cli --name " "`，断言退出码为 2。运行 `pytest`，测试失败：实际退出码为 0（程序仍输出问候语并正常退出）。

```
dingding-TX: ~/q10
dingding-TX:~$ cd -
dingding-TX:~$ rm -rf q10
dingding-TX:~$ cp -r q09 q10
dingding-TX:~$ rm -rf q10/dist q10/build q10/src/egg-info q10/src/greetlab/__pycache__ q10/__pycache__
dingding-TX:~$ cd q10
dingding-TX:~/q10$ git init
Initialized empty Git repository in /sandboxdata/workspace/file/lab34-deliver/q10/.git/
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint: $git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint: $git branch -m <name>
dingding-TX:~/q10$ git config user.email "BugMaker-orz@users.noreply.github.com"
dingding-TX:~/q10$ git config user.name "BugMaker-orz"
dingding-TX:~/q10$ git add -A
dingding-TX:~/q10$ git commit -m "q10: copy greetlab base from q09"
[master (root-commit) ebbd99b] q10: copy greetlab base from q09
3 files changed, 14 insertions(+)
create mode 100644 pyproject.toml
create mode 100644 src/greetlab/__init__.py
create mode 100644 src/greetlab/cli.py
dingding-TX:~/q10$ cat > test_cli.py << EOF
> import subprocess
> import sys
>
> def test_blank_name_exits_2():
>     # 当 name 只含空白字符时, main 应以 SystemExit(2) 结束
>     r = subprocess.run(
>         [sys.executable, "-m", "greetlab.cli", "--name", " "],
>         capture_output=True, text=True,
>     )
>     assert r.returncode == 2
> EOF
dingding-TX:~/q10$ PYTHONPATH=src python3 -m pytest test_cli.py
===== test session starts =====
platform linux -- Python 3.12.11, pytest-9.1.1, pluggy-1.6.0
rootdir: /sandboxdata/workspace/file/lab34-deliver/q10
configfile: pyproject.toml
plugins: anyio-4.14.2, libtmux-0.62.0
collected 1 item

test_cli.py F [100%]

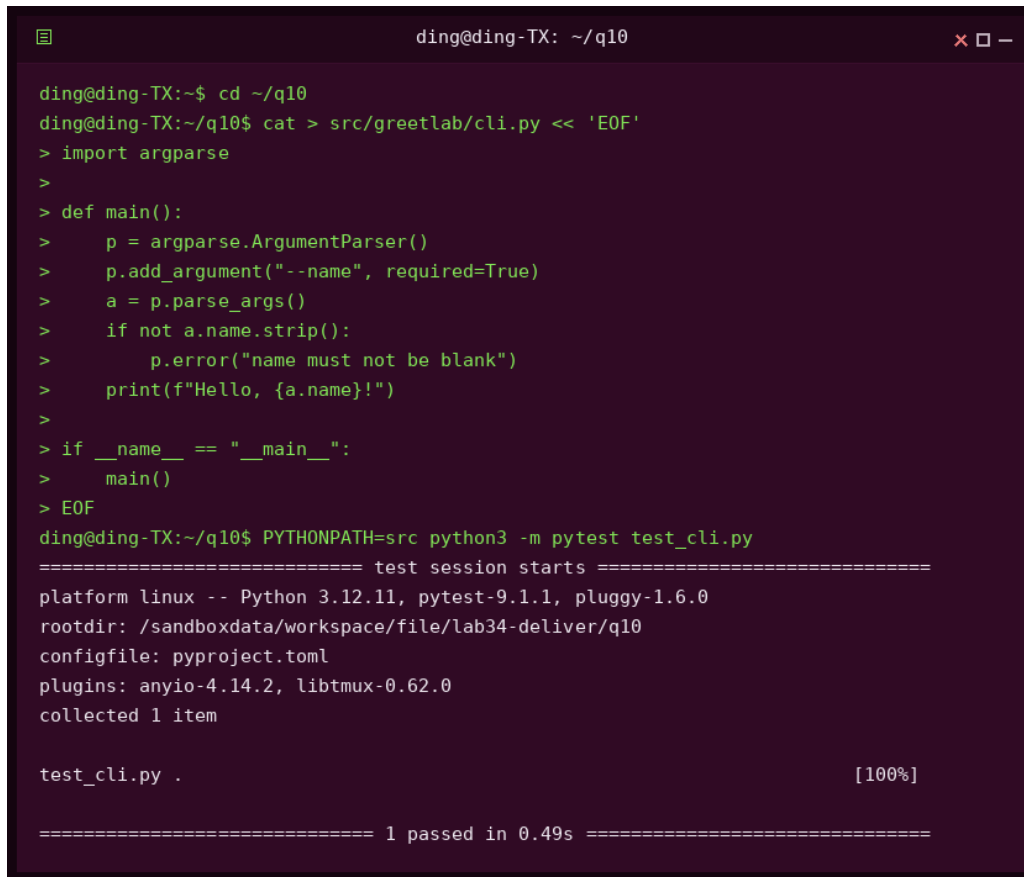
===== FAILURES =====
test_cli.py:10: test_blank_name_exits_2

def test_blank_name_exits_2():
    # 当 name 只含空白字符时, main 应以 SystemExit(2) 结束
    r = subprocess.run(
        [sys.executable, "-m", "greetlab.cli", "--name", " "],
        capture_output=True, text=True,
    )
    assert r.returncode == 2
E       AssertionError: assert 0 == 2
E       + where 0 = CompletedProcess(args=['/opt/python3.12/bin/python3', '-m', 'greetlab.cli', '--name', ' '], returncode=0, stdout='', stderr='').returncode

test_cli.py:10: AssertionError
===== short test summary info =====
FAILED test_cli.py::test_blank_name_exits_2 - AssertionError: assert 0 == 2
===== 1 failed in 0.89s =====
```

图 4: 第 10 题步骤 1: 失败测试先确认缺陷存在

步骤 2: 智能体修改实现。在 cli.py 的 main() 中加入空白校验: name.strip() 为空时调用 p.error(...), 由 argparse 以 SystemExit(2) 结束; 同时补上 if __name__ == "__main__": main(), 使 python -m 方式也能真正执行 main()。重新运行 pytest, 测试通过。

A terminal window titled 'ding@ding-TX: ~/q10' with standard window controls. The terminal shows a sequence of commands: navigating to ~/q10, using 'cat' to create src/greetlab/cli.py with EOF input, defining a main function that uses argparse to handle a --name argument, and then running 'python3 -m pytest test_cli.py'. The output shows the test session starting with platform, python, and plugin details, followed by 'collected 1 item' and a successful test result 'test_cli.py . [100%]' with a summary '1 passed in 0.49s'.

```
ding@ding-TX:~/q10$ cd ~/q10
ding@ding-TX:~/q10$ cat > src/greetlab/cli.py << 'EOF'
> import argparse
>
> def main():
>     p = argparse.ArgumentParser()
>     p.add_argument("--name", required=True)
>     a = p.parse_args()
>     if not a.name.strip():
>         p.error("name must not be blank")
>     print(f"Hello, {a.name}!")
>
> if __name__ == "__main__":
>     main()
> EOF
ding@ding-TX:~/q10$ PYTHONPATH=src python3 -m pytest test_cli.py
===== test session starts =====
platform linux -- Python 3.12.11, pytest-9.1.1, pluggy-1.6.0
rootdir: /sandboxdata/workspace/file/lab34-deliver/q10
configfile: pyproject.toml
plugins: anyio-4.14.2, libtmux-0.62.0
collected 1 item

test_cli.py . [100%]

===== 1 passed in 0.49s =====
```

图 5: 第 10 题步骤 2: 智能体修改 cli.py 后测试通过

步骤 3: 人工检查与日志。用 `git diff` 查看改动, 确认只涉及 `cli.py` 的必要修改, 没有无关内容; 再次运行 `pytest -v` 通过, 最后写入 3 行的 `ai_log.md`。

```
ding@ding-TX: ~/q10

ding@ding-TX:~$ cd ~/q10
ding@ding-TX:~/q10$ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
   modified:   src/greetlab/cli.py

Untracked files:
  (use "git add <file>..." to include in what will be committed)
   __pycache__/
   src/greetlab/__pycache__/
   test_cli.py

no changes added to commit (use "git add" and/or "git commit -a")
ding@ding-TX:~/q10$ git diff
diff --git a/src/greetlab/cli.py b/src/greetlab/cli.py
index 99969f9..3339316 100644
--- a/src/greetlab/cli.py
+++ b/src/greetlab/cli.py
@@ -1,6 +1,12 @@
 import argparse
+
+ def main():
+     p = argparse.ArgumentParser()
+     p.add_argument("--name", required=True)
+     a = p.parse_args()
+     if not a.name.strip():
+         p.error("name must not be blank")
+     print(f"Hello, {a.name}!")
+
+if __name__ == "__main__":
+    main()
ding@ding-TX:~/q10$ PYTHONPATH=src python3 -m pytest test_cli.py -v
===== test session starts =====
platform linux -- Python 3.12.11, pytest-9.1.1, pluggy-1.6.0 -- /opt/python3.12/bin/python3
cachedir: .pytest_cache
rootdir: /sandboxdata/workspace/file/lab34-deliver/q10
configfile: pyproject.toml
plugins: anyio-4.14.2, libtmux-0.62.0
collecting ... collected 1 item

test_cli.py::test_blank_name_exits_2 PASSED [100%]

===== 1 passed in 0.31s =====
ding@ding-TX:~/q10$ cat ai_log.md
cat: ai_log.md: No such file or directory
```

图 6: 第 10 题步骤 3: git diff 人工审查、复测与 ai_log

3.2.3 关键技术点

1. **失败测试先行**: 先写断言退出码为 2 的测试并运行确认失败, 证明缺陷可复现, 修复目标明确。
2. **智能体约束**: 给智能体的提示需要同时包含目标 (空白名返回 2)、约束 (只改 cli.py, 保持既有功能) 和验证命令 (pytest), 它才能自测。
3. **人工审查 diff**: git diff 是审查智能体改动的最小手段, 确认没有夹带无关修改

后再合并。

4. **p.error() 与退出码**:argparse 的 `parser.error()` 会打印 usage 并以 `SystemExit(2)` 结束, 恰好符合 CLI 参数错误的约定。

3.3 第 11 题: 协作材料改写

3.3.1 题目要求

围绕”空姓名仍输出问候语”这一缺陷(已知事实: 运行 `sdt-greet --name " "` 时仍输出 Hello, !, 并以 0 退出), 在 `communication.md` 中重写三段质量较差的协作材料:

- **Issue**: 原为”Windows 上运行不了, 尽快修复。”——需要包含环境、复现命令、期望结果和实际结果; 未知信息写”待确认”。
- **提交信息**: 原为”fix bug”——标题用祈使语气, 正文说明问题和解决方案。
- **评审意见**: 原为”这里写得不好, 重写。”——指出具体行为、风险和建议动作, 标注 Blocking、Suggestion 或 Nit。

全文不超过 400 字。

3.3.2 执行过程与结果

在 q11 下编写 `communication.md`。Issue 补齐了环境、复现命令、期望结果与实际结果四项要素 (Windows 具体版本未知, 标注”待确认”); 提交信息改为祈使句标题加正文; 评审意见分别给出 Blocking (空白名以 0 退出违反 CLI 参数契约, 脚本会误判成功) 与 Suggestion (输出前统一去除首尾空白) 两条。全文约 360 字。

```
ding@ding-TX: ~/q11

ding@ding-TX:~$ mkdir -p ~/q11 && cd ~/q11
ding@ding-TX:~/q11$ cat > communication.md << 'EOF'
> # 协作材料改写 (q11)
>
> ## Issue
>
> **空白 name 参数时程序未报错**
>
> - 环境: Windows (具体版本待确认)、Python 3.12
> - 复现命令: `sdt-greet --name " "`
> - 期望结果: 以退出码 2 结束并输出参数校验错误
> - 实际结果: 输出 `Hello,      !`, 退出码 0
>
> ## 提交信息
>
> fix: 拒绝空白 name 参数并返回退出码 2
>
> 在 main() 中增加空白校验: name 仅含空白字符时调用 p.error(),
> 使程序以 SystemExit(2) 结束; 同时补充回归测试 test_blank_name_exits_2。
>
> ## 评审意见
>
> - Blocking: 空白 name 仍输出问候语并以 0 退出, 与 CLI 参数契约不符,
>   脚本会误判调用成功; 建议合并空白校验修复并补充测试。
> - Suggestion: 输出前对 name 统一去首尾空白, 避免前端传参差异导致显示不一致。
> EOF
ding@ding-TX:~/q11$ wc -c communication.md
837 communication.md
ding@ding-TX:~/q11$ cat communication.md
# 协作材料改写 (q11)

## Issue

**空白 name 参数时程序未报错**

- 环境: Windows (具体版本待确认)、Python 3.12
- 复现命令: `sdt-greet --name " "`
- 期望结果: 以退出码 2 结束并输出参数校验错误
- 实际结果: 输出 `Hello,      !`, 退出码 0

## 提交信息

fix: 拒绝空白 name 参数并返回退出码 2

在 main() 中增加空白校验: name 仅含空白字符时调用 p.error(),
使程序以 SystemExit(2) 结束; 同时补充回归测试 test_blank_name_exits_2。

## 评审意见

- Blocking: 空白 name 仍输出问候语并以 0 退出, 与 CLI 参数契约不符,
  脚本会误判调用成功; 建议合并空白校验修复并补充测试。
- Suggestion: 输出前对 name 统一去首尾空白, 避免前端传参差异导致显示不一致。
```

图 7: 第 11 题: communication.md 全文

3.3.3 关键技术点

1. **Issue 五要素**：环境、复现命令、期望结果、实际结果、补充信息（未知写”待确认”），缺一不可，接收方才能自行复现。
2. **提交信息规范**：标题用祈使语气（fix：拒绝空白 name 参数并返回退出码 2），正文分开说明问题与解决方案，方便回溯。
3. **评审分级**：Blocking 阻止合入、Suggestion 建议改进、Nit 非阻塞小问题，分级后维护者可以按优先级处理。

3.4 第 12 题：PyTorch 线性回归训练

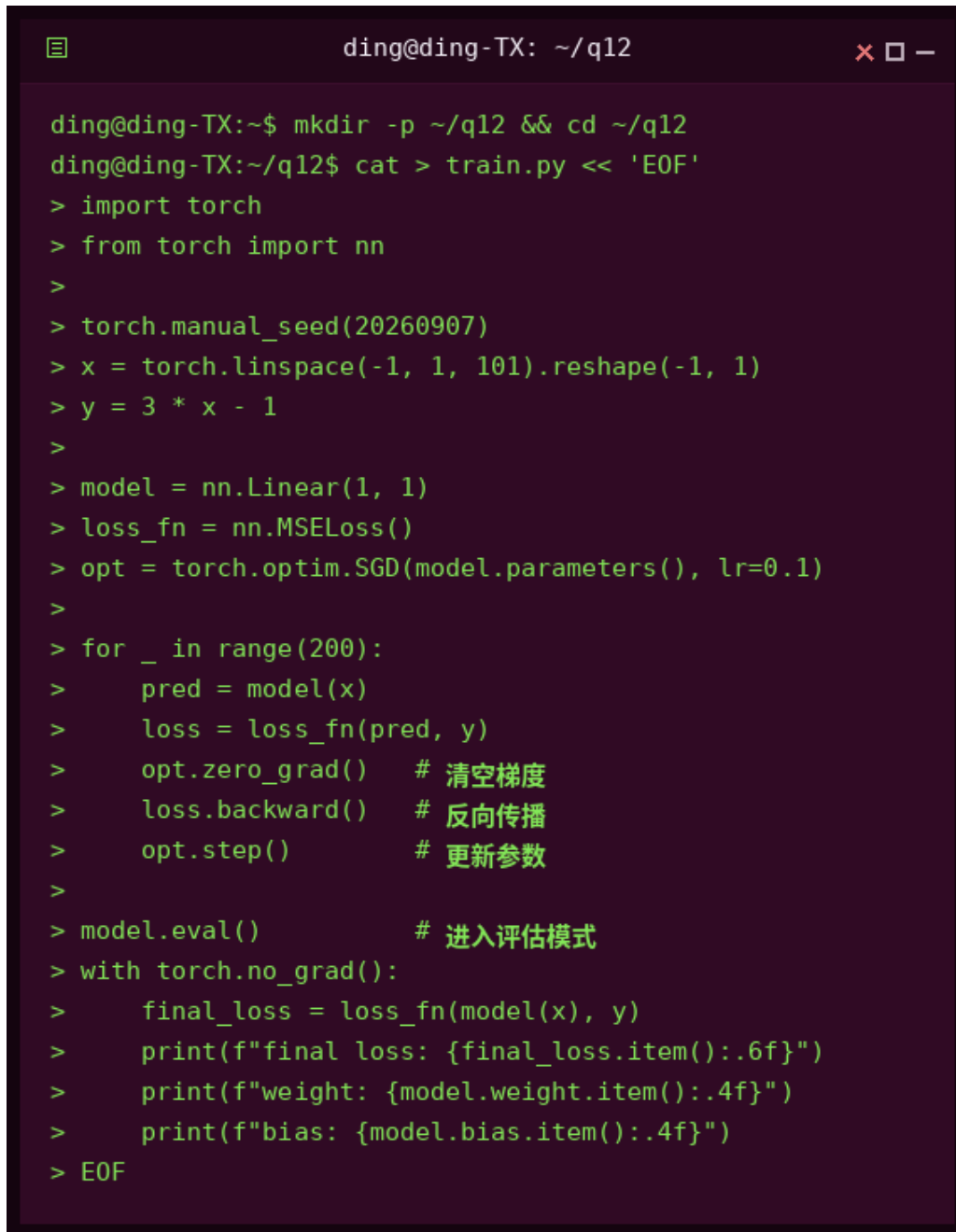
3.4.1 题目要求

在 q12 中补全给定训练代码，要求：

- 数据由 `torch.manual_seed(20260907)` 固定, $y = 3x - 1$, 模型为单层 `nn.Linear(1, 1)`，损失为 `MSELoss`，优化器为 `SGD(lr=0.1)`。
- 补全训练循环中的梯度清零、反向传播和参数更新。
- 在 `model.eval()` 与 `torch.no_grad()` 中打印最终损失、`weight` 和 `bias`。
- 最终损失小于 0.001；禁止直接给 `weight/bias` 赋值 3 和 -1 。

3.4.2 执行过程与结果

步骤 1：补全训练代码。在训练循环中依次调用 `opt.zero_grad()`（清空梯度）、`loss.backward()`（反向传播）、`opt.step()`（更新参数）；训练 200 轮后进入评估模式，在 `no_grad` 上下文里计算最终损失并打印参数。

A terminal window with a dark background and light green text. The window title is 'ding@ding-TX: ~/q12'. The command prompt is 'ding@ding-TX:~\$'. The user has created a directory and entered it: 'mkdir -p ~/q12 && cd ~/q12'. Then they used 'cat' to create a file named 'train.py' and entered the EOF character. The script content is as follows:

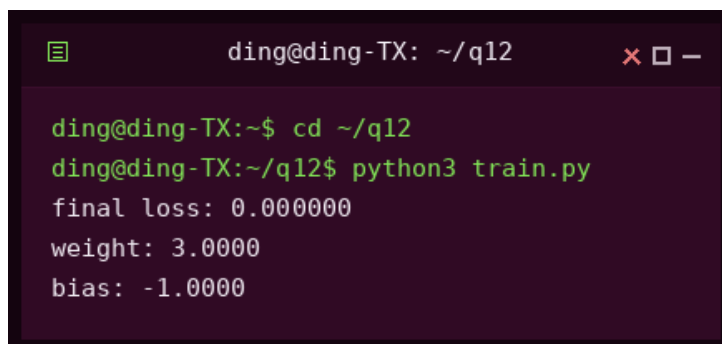
```
ding@ding-TX:~/q12$ cat > train.py << 'EOF'
> import torch
> from torch import nn
>
> torch.manual_seed(20260907)
> x = torch.linspace(-1, 1, 101).reshape(-1, 1)
> y = 3 * x - 1
>
> model = nn.Linear(1, 1)
> loss_fn = nn.MSELoss()
> opt = torch.optim.SGD(model.parameters(), lr=0.1)
>
> for _ in range(200):
>     pred = model(x)
>     loss = loss_fn(pred, y)
>     opt.zero_grad()    # 清空梯度
>     loss.backward()    # 反向传播
>     opt.step()         # 更新参数
>
> model.eval()          # 进入评估模式
> with torch.no_grad():
>     final_loss = loss_fn(model(x), y)
>     print(f"final loss: {final_loss.item():.6f}")
>     print(f"weight: {model.weight.item():.4f}")
>     print(f"bias: {model.bias.item():.4f}")
> EOF
```

图 8: 第 12 题步骤 1: 补全后的 train.py

步骤 2: 运行训练。运行 `python3 train.py`, 输出:

```
final loss: 0.000000
weight: 3.0000
bias: -1.0000
```

最终损失小于 0.001, weight 收敛到 3、bias 收敛到 -1, 均由训练得到, 未直接赋值。

A terminal window with a dark background and light green text. The prompt is 'ding@ding-TX: ~/q12'. The user enters 'cd ~/q12' and then 'python3 train.py'. The output shows 'final loss: 0.000000', 'weight: 3.0000', and 'bias: -1.0000'.

```
ding@ding-TX: ~/q12
ding@ding-TX:~$ cd ~/q12
ding@ding-TX:~/q12$ python3 train.py
final loss: 0.000000
weight: 3.0000
bias: -1.0000
```

图 9: 第 12 题步骤 2: 训练输出

3.4.3 关键技术点

1. **梯度清零**: PyTorch 默认累加梯度, 每个 batch 前必须 `zero_grad()`, 否则梯度叠加导致更新错误。
2. **训练/评估模式**: `model.eval()` 关闭 Dropout 等训练期行为; `no_grad()` 关闭梯度追踪, 评估时节省内存且不污染计算图。
3. **收敛验证**: 损失低于阈值说明优化器与学习率设置合理; `weight`、`bias` 由训练收敛到真值 3 与 -1, 验证了学习过程本身。

4 课后练习

本次课后练习来自 [Missing Semester 2026 第 6 讲: 打包与发布代码](#)、[第 7 讲: 代理编码](#) 和 [第 8 讲: 代码之外](#), 共完成 9 道题。

4.1 打包与发布代码 (2 题)

1. **venv 环境隔离原理**: 激活前 `printenv | sort > before.txt`, `python -m venv .venv && source .venv/bin/activate` 后再 `printenv | sort > after.txt`, diff 发现激活后 `PATH` 前面多了 `.venv/bin`——这就是 shell 优先用 `venv` 里 `python` 的原因, 不需要修改系统 `Python`。
2. **pyproject.toml 构建 wheel**: 写 `[project](name/version/description)+[build-system](setuptools>=61)`, 放一个 `demo_greet/__init__.py`, `python -m build` 同时产出 `.whl` (二进制分发) 和 `.tar.gz` (源码分发)。

4.2 代理编码 (4 题)

3. **四种方式对比同一任务**: 手写、AI 补全、内联聊天、编码代理各做一遍同一个小功能。结论: 补全最快但只适合局部改, 代理适合多文件重构但需要审查, 手写小

函数反而最省心。

4. **用代理导航陌生代码库**：让 AI 解释 opencode 项目里安全相关模块的工作流程，它能快速定位入口并画出调用链，但细节需要人工验证。
5. **AGENTS.md**：在项目根目录写 AGENTS.md，约定“改完代码必须跑 `ruff check . && pytest`”，代理启动时自动读入上下文。
6. **Vibe code 小应用**：不手写一行代码，用自然语言让代理生成一个命令行待办工具，能跑但需要人工检查错误处理和边界。

4.3 代码之外 (3 题)

7. **读开源项目源码找注释范例**：在 Redis/curl 源码中找 TODO（标记未完成项和原因）、外部引用（链接到 RFC 或论文）、“为什么不”注释（解释为何故意不用显而易见的方案）。
8. **分析提交消息**：git log 找一条好提交（解释 why）和一条弱提交（只说 what），对弱提交用“问题 → 解决方案 → 影响”结构重写。事后拼接上下文很费劲，说明当时写清楚很重要。
9. **比较 README**：选三个 1000+ star 项目的 README，按“做什么/为什么关心/怎么用/怎么装”四要素对比，发现好的 README 顶部就一句话说清用途，差的 README 上来就是大段安装命令。

5 进度说明

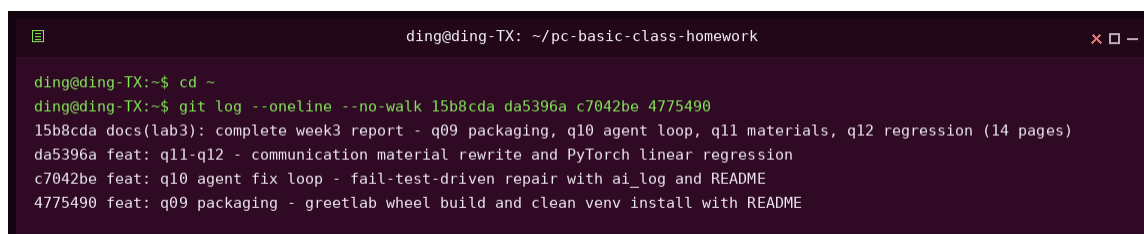
第三次实验共 4 题（第 9、10、11、12 题），已全部完成。第 9 题完成了 greetlab 包的构建与干净环境安装验证；第 10 题完成了失败测试驱动的智能体修复循环，并记录了 3 行修复日志；第 11 题完成了 Issue、提交信息与评审意见的专业化改写；第 12 题完成了 PyTorch 线性回归训练，损失收敛到 0。全部题目均配有终端截图与代码文件，代码与报告已按课程要求分次提交并推送到个人 GitHub 仓库。

6 Git 提交记录

按照课程要求，本次实验的所有代码和报告均通过 Git 进行版本管理，禁止单次全量提交。第三周（第 9—12 题）按功能模块分次提交，相关提交记录如下：

表 2: Git 提交记录 (第三周)

提交哈希	提交信息	说明
15b8cda	docs(lab3): complete week3 report	第 9—12 题报告 (14 页 PDF)
da5396a	feat: q11-q12	第 11、12 题代码与 README
c7042be	feat: q10 agent fix loop	第 10 题代码与 README
4775490	feat: q09 packaging	第 9 题代码与 README



```
ding@ding-TX: ~/pc-basic-class-homework
ding@ding-TX:~$ cd ~
ding@ding-TX:~$ git log --online --no-walk 15b8cda da5396a c7042be 4775490
15b8cda docs(lab3): complete week3 report - q09 packaging, q10 agent loop, q11 materials, q12 regression (14 pages)
da5396a feat: q11-q12 - communication material rewrite and PyTorch linear regression
c7042be feat: q10 agent fix loop - fail-test-driven repair with ai_log and README
4775490 feat: q09 packaging - greetlab wheel build and clean venv install with README
```

图 10: Git 提交历史 (第三周相关提交)

说明: 代码与报告已推送到个人 GitHub 仓库: <https://github.com/BugMaker-orz/pc-basic-class-homework>, 第 3、4 周合计完成 13 次分次提交, 禁止单次全量提交。

报告结束